

Fachinformatiker für Anwendungsentwicklung
Dokumentation zur schulischen Projektarbeit

Erweiterung von „Yggdrasil’s Depths“ mit Graphical User Interface

Abgabetermin: Twistringen, den 05.07.2026

Auszubildender:

Bastian Rentzsch
Zitadelle 1
27239 Twistringen



Ausbildungsbetrieb:

cbm GbmH
Wegesende 3-4
28195 Bremen

Inhaltsverzeichnis

Abbildungsverzeichnis	III
Tabellenverzeichnis	IV
Listings	V
Abkürzungsverzeichnis	VI
1 Einleitung	1
1.1 Projektumfeld	1
1.2 Projektziel	1
1.3 Projektbegründung	1
1.4 Projektschnittstellen	1
1.5 Projektabgrenzung	2
2 Projektplanung	2
2.1 Projektphasen	2
2.2 Ressourcenplanung	3
2.3 Entwicklungsprozess	3
3 Analysephase	4
3.1 Ist-Analyse	4
3.2 Wirtschaftlichkeitsanalyse	4
3.2.1 „Make or Buy“-Entscheidung	4
3.2.2 Projektkosten	4
3.2.3 Amortisationsdauer	5
3.3 Nutzwertanalyse	5
3.4 Anwendungsfälle	6
4 Entwurfsphase	7
4.1 Zielplattform	7
4.2 Architekturdesign	7
4.3 Entwurf der Benutzeroberfläche	7
4.4 Datenmodell	8
4.5 Geschäftslogik	8
4.6 Maßnahmen zur Qualitätssicherung	8
5 Implementierungsphase	9
5.1 Implementierung der Benutzeroberfläche	9

5.2	Implementierung der Geschäftslogik	9
6	Dokumentation	10
6.1	Entwicklerdokumentation	10
7	Fazit	10
7.1	Soll-/Ist-Vergleich	10
7.2	Lessons Learned	11
7.3	Ausblick	11
A	Anhang	i
A.1	Detaillierte Zeitplanung	i
A.2	Oberflächenentwürfe	ii
A.3	Screenshots der Anwendung	v
A.4	Entwicklerdokumentation	ix
A.5	Klasse: InventoryScreen	xv
A.6	Klassendiagramm der grafischen Benutzeroberfläche	xix
A.7	Sequenzdiagramm	xx

Abbildungsverzeichnis

1	Use Case-Diagramm	6
2	GUI-Mock-ups des Hauptmenüs, des Optionsmenüs, des Fensters zur Erstellung eines neuen Spiels sowie des Spielfensters.	ii
3	GUI-Mock-ups des Pausenmenüs und seiner Untermenüs: Status, Inventar, Optionen sowie Ausrüstungsauswahl.	iii
4	GUI-Mock-ups der Kampfansicht sowie des Angriffs- und des Itemauswahlmenüs.	iv
5	Hauptmenü	v
6	Fenster zur Erstellung eines neuen Spiels	vi
7	Spielfenster während der Bewegung im Dungeon	vi
8	Status des Spielers	vii
9	Inventar des Spielers	vii
10	Spielfenster während eines Kampfes	viii
11	Klassendiagramm der grafischen Oberfläche	xix
12	Sequenzdiagramm	xx

Tabellenverzeichnis

1	Grobe Zeitplanung	2
2	Nutzwertanalyse	5
3	Soll-/Ist-Vergleich	11

Listings

1	Klasse: InventoryScreen	xv
---	-----------------------------------	----

Abkürzungsverzeichnis

GUI	Graphical User Interface
JVM	Java Virtual Machine
JMX	Java Management Extensions
REST	Representational State Transfer
API	Application Programming Interface
IDE	Integrated Development Environment
MVC	Model View Controller
UML	Unified Modeling Language

1 Einleitung

1.1 Projektumfeld

Auftraggeber und Kunde des Projekts bin ich selbst. Das Projekt wird als eigenständiges Entwicklungsprojekt im Rahmen des Unterrichts durchgeführt.

1.2 Projektziel

Ziel des Projekts ist die Erweiterung des bestehenden Projekts „Yggdrasil’s Depths“ um eine grafischen Benutzeroberfläche. Dadurch soll die Bedienbarkeit verbessert und die im Unterricht vermittelten Kenntnisse im Bereich der GUI-Entwicklung praktisch angewendet und vertieft werden.

1.3 Projektbegründung

Die Motivation hinter diesem Projekt besteht darin, die im Unterricht erworbenen Kenntnisse im Bereich der GUI praktisch anzuwenden und zu vertiefen. Ziel ist es, die theoretischen Inhalte durch die Entwicklung einer benutzerfreundlichen und funktionalen Oberfläche in einem realen Projekt umzusetzen sowie praktische Erfahrungen in der Konzeption und Implementierung von GUIs zu sammeln.

1.4 Projektschnittstellen

Die Anwendung greift auf mehrere .dat-Dateien im Ressourcenverzeichnis des Projekts zu. Diese dienen als Datenbasis für den Spielbetrieb:

- savedata.dat – enthält den aktuellen Speicherstand des Benutzers.
- itemcodex.dat – enthält sämtliche im Spiel verfügbaren Gegenstände.
- enemycodex.dat – enthält alle im Spiel vorhandenen Gegner.

Nach Abschluss des Projekts wird das Ergebnis im Rahmen einer Präsentation vor der Klasse vorgestellt.

1.5 Projektabgrenzung

Folgende Punkte sind ausdrücklich nicht Bestandteil des Projekts:

- Anbindung an externe Datenbanksysteme (PostgreSQL, MySQL)
- Überwachung externer JVM-Prozesse via JMX
- Netzwerkfähigkeit oder REST-API

2 Projektplanung

2.1 Projektphasen

Die Projektdurchführung erfolgt im Zeitraum vom 22.06.2026 bis zum 05.07.2026. Für die Umsetzung stehen insgesamt zwei Arbeitswochen mit einer 40-Stunden-Woche und einer täglichen Arbeitszeit von acht Stunden zur Verfügung.

Projektphase	Geplante Zeit
Analysephase	6 h
Entwurfsphase	12 h
Implementierungsphase	40 h
Erstellen der Dokumentation	22 h
Gesamt	80 h

Tabelle 1: Grobe Zeitplanung

Eine detailliertere Zeitplanung findet sich im Anhang A.1: Detaillierte Zeitplanung auf Seite i.

2.2 Ressourcenplanung

Hardware

- Arbeitsplatz mit Thin-Client

Software

- IntelliJ IDEA 2026.1 – Entwicklungsumgebung (IDE)
- Eclipse 2026-03 – Entwicklungsumgebung (IDE)
- Oracle OpenJDK 21 - Java Development Kit (JDK)
- Windows 11 Pro – Betriebssystem
- Github – Versionsverwaltung
- diagrams.net – Programm zur Erstellung von Diagrammen und Modellen
- PlantUML – Programm zur Erstellung von Diagrammen und Modellen
- TeXstudio – Entwicklungsumgebung von $\text{\LaTeX}$
- MiKTeX – Distribution des Textsatzsystems $\text{\LaTeX}$

Personal

- Auszubildender zum Fachinformatiker für Anwendungsentwicklung – Planung, Implementierung, Test und Dokumentation des Projekts

2.3 Entwicklungsprozess

Für die Umsetzung des Projekts wird ein iterativer Entwicklungsprozess verfolgt. Da das Projekt von einer einzelnen Person durchgeführt wird und sich der Funktionsumfang während der Entwicklung schrittweise überprüfen lässt, eignet sich dieses Vorgehensmodell besonders.

Die grafische Benutzeroberfläche wird in mehreren aufeinanderfolgenden Iterationen entwickelt. Nach der Implementierung einzelner Funktionen werden diese unmittelbar getestet und bei Bedarf angepasst oder erweitert. Dadurch können Fehler frühzeitig erkannt und behoben sowie die Benutzeroberfläche kontinuierlich verbessert werden.

Ein formales agiles Vorgehensmodell, wie beispielsweise Scrum, wird aufgrund des geringen Projektumfangs und der Durchführung durch eine einzelne Person nicht eingesetzt.

3 Analysephase

3.1 Ist-Analyse

Das bestehende Projekt „Yggdrasil’s Depths“ verfügt bereits über eine vollständige Spiellogik, wird jedoch ausschließlich über die Kommandozeile bedient. Die Bedienung erfolgt durch Texteingaben, wodurch die Benutzerfreundlichkeit eingeschränkt ist und der Spielfluss unterbrochen werden kann.

Ziel des Projekts ist die Entwicklung einer Graphical User Interface (GUI), die die bestehende Spiellogik erweitert und bei Bedarf diese anpassen. Dadurch soll die Anwendung intuitiver bedienbar werden und eine verbesserte Benutzererfahrung bieten.

3.2 Wirtschaftlichkeitsanalyse

Da es sich um ein Lernprojekt handelt und der Auftraggeber zugleich der Entwickler ist, steht nicht die wirtschaftliche Gewinnerzielung im Vordergrund. Stattdessen liegt der Schwerpunkt auf dem Erwerb praktischer Erfahrungen in der Entwicklung grafischer Benutzeroberflächen sowie auf der Verbesserung der Bedienbarkeit der bestehenden Anwendung.

3.2.1 „Make or Buy“-Entscheidung

Eine bestehende Software, die die Anforderungen dieses Projekts erfüllt und gleichzeitig auf der vorhandenen Spiellogik aufbaut, ist nicht verfügbar. Der Einsatz eines externen Produkts würde eine vollständige Neuentwicklung oder umfangreiche Anpassungen erfordern.

Aus diesem Grund wird die grafische Benutzeroberfläche eigenständig entwickelt und direkt in das bestehende Projekt integriert.

3.2.2 Projektkosten

Die Kosten für die Durchführung des Projekts setzen sich aus Personal- und Ressourcenkosten zusammen.

Für die Kalkulation wird ein fiktiver kalkulatorischer Stundensatz von 13,90 € pro Stunde für den Auszubildenden angesetzt. Zusätzlich werden für die Nutzung von Ressourcen (z. B. Arbeitsplatz, Softwareumgebung und Infrastruktur) pauschal 15 € pro Stunde berücksichtigt.

Die geschätzte Projektdauer beträgt insgesamt 80 Stunden.

3 Analysephase

Die Gesamtkosten ergeben sich wie folgt:

Personalkosten:

$$80h * 13,90 \text{ €} = 1.112,00 \text{ €} \quad (1)$$

Ressourcenkosten:

$$80h * 15,00 \text{ €} = 1.200,00 \text{ €} \quad (2)$$

Gesamtkosten:

$$1.112,00 \text{ €} + 1.200,00 \text{ €} = 2.312,00 \text{ €} \quad (3)$$

3.2.3 Amortisationsdauer

Eine klassische Amortisationsrechnung ist für dieses Projekt nicht sinnvoll, da keine direkten finanziellen Einsparungen oder Umsätze erzielt werden.

Der Nutzen besteht insbesondere in der Verbesserung der Benutzerfreundlichkeit sowie im Erwerb praktischer Kenntnisse in der GUI-Entwicklung. Darüber hinaus kann die entwickelte Benutzeroberfläche als Grundlage für zukünftige Erweiterungen des Projekts dienen.

3.3 Nutzwertanalyse

Durch die Einführung einer grafischen Benutzeroberfläche ergeben sich gegenüber der bisherigen Konsolenanwendung mehrere qualitative Vorteile:

Kriterium	Vorher	Nachher
Benutzerfreundlichkeit	gering	hoch
Übersichtlichkeit	gering	hoch
Bedienkomfort	gering	hoch
Erweiterbarkeit der Oberfläche	eingeschränkt	gut
Lernwert	gering	hoch

Tabelle 2: Nutzwertanalyse

Die GUI verbessert insbesondere die Bedienbarkeit und erhöht die Attraktivität der Anwendung für den Benutzer.

3.4 Anwendungsfälle

Die Anwendung unterstützt folgende Anwendungsfälle:

- Starten eines neuen Spiels
- Laden eines vorhandenen Spielstands
- Speichern des aktuellen Spielstands
- Navigation durch die Spieloberfläche
- Anzeigen der Spielerinformationen
- Durchführung von Kämpfen
- Anzeigen von Gegner- und Iteminformationen
- Beenden der Anwendung

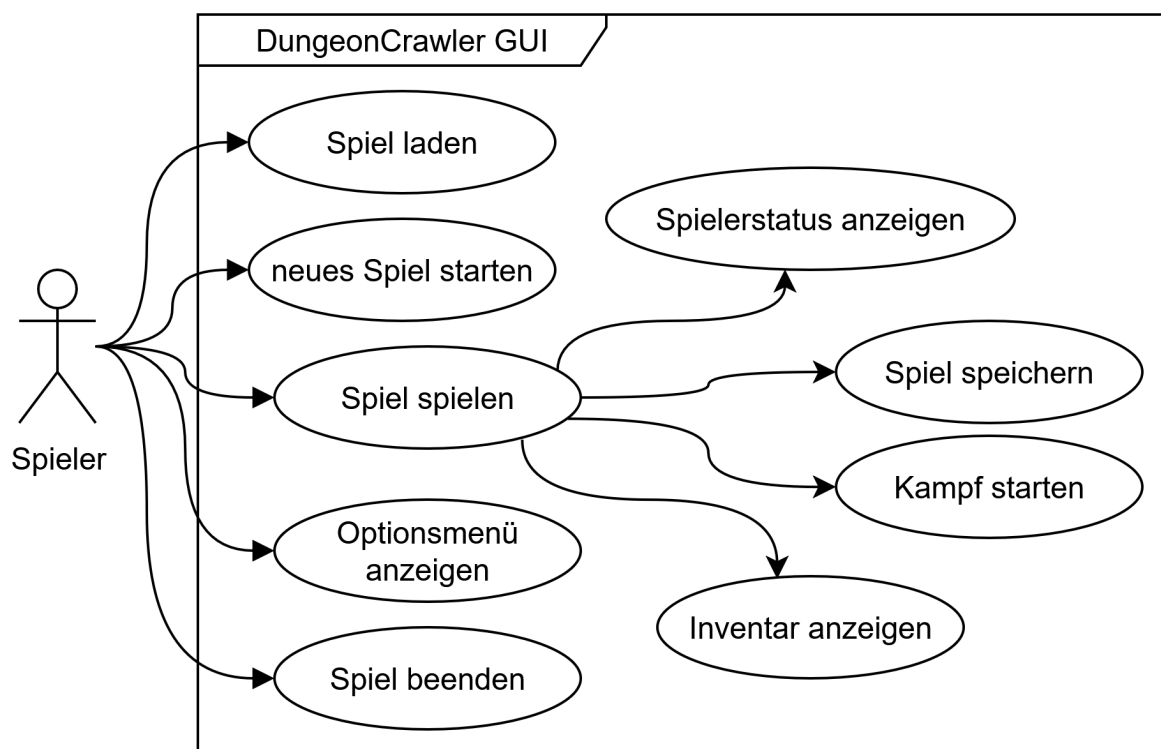


Abbildung 1: Use Case-Diagramm

4 Entwurfsphase

4.1 Zielplattform

Die Anwendung wird als Desktop-Anwendung für Windows entwickelt. Als Programmiersprache wird Java verwendet, da das bestehende Projekt bereits in Java implementiert wurde und somit eine nahtlose Erweiterung der vorhandenen Spiellogik möglich ist.

Die Entwicklung erfolgt mit IntelliJ IDEA, Eclipse und dem Oracle OpenJDK 21. Da sämtliche Spieldaten in lokalen .dat-Dateien gespeichert werden, wird keine Datenbank benötigt.

4.2 Architekturdesign

Für die Erweiterung wird das MVC-Prinzip angewendet. Die vorhandene Spiellogik bildet das Model, während die neu entwickelte grafische Benutzeroberfläche die View darstellt. Der Controller übernimmt die Verarbeitung der Benutzereingaben und vermittelt zwischen Oberfläche und Spiellogik.

Durch diese Trennung bleiben Spiellogik und Benutzeroberfläche unabhängig voneinander. Änderungen an der GUI können vorgenommen werden, ohne die Spiellogik wesentlich anzupassen.

4.3 Entwurf der Benutzeroberfläche

Die Anwendung erhält eine grafische Benutzeroberfläche, welche die bisherige Konsolenoberfläche ersetzt. Ziel ist eine intuitive und übersichtliche Bedienung.

Die Oberfläche besteht aus einem Hauptmenü mit den Funktionen „neues Spiel starten“, „Spiel laden“, „Optionen“ und „Spiel beenden“. Während des Spiels werden der aktuelle Spielerstatus, das Inventar sowie die verfügbaren Aktionen übersichtlich dargestellt. Dialogfenster informieren den Benutzer über Kampfergebnisse oder Fehlermeldungen.

Die Gestaltung orientiert sich an den Grundprinzipien der Usability. Häufig genutzte Funktionen sind leicht erreichbar und die Navigation erfolgt einheitlich über Schaltflächen und Menüs. Ein Mock-up der Benutzeroberfläche befindet sich im Anhang A.2: Oberflächenentwürfe auf Seite ii.

4.4 Datenmodell

Das Projekt wird um drei Datendateien erweitert: savedata.dat für den Spielstand, itemcodex.dat für alle verfügbaren Gegenstände und enemycodex.dat für alle Gegner. Die GUI greift dabei nur lesend und schreibend auf diese Dateien zu.

4.5 Geschäftslogik

Die bestehende Spiellogik soll größtenteils unverändert weiterverwendet und durch die grafische Benutzeroberfläche gesteuert werden.

Der Benutzer löst Aktionen über die GUI aus. Diese werden vom Controller verarbeitet und an die Spiellogik weitergeleitet. Die Ergebnisse werden anschließend in der Benutzeroberfläche dargestellt, sodass die Spiellogik von der Darstellung getrennt bleibt.

Ein Klassendiagramm der grafischen Benutzeroberfläche befindet sich im Anhang A.6: Klassendiagramm der grafischen Benutzeroberfläche auf Seite xix. Die möglichen Abläufe des Projekts werden in einem Sequenzdiagramm dargestellt, dass sich im Anhang A.7: Sequenzdiagramm auf Seite xx befindet.

4.6 Maßnahmen zur Qualitätssicherung

Zur Sicherstellung der Funktionsfähigkeit werden die implementierten Funktionen nach jeder Entwicklungsiteration getestet. Dabei werden insbesondere das Starten und Laden eines Spiels, das Speichern des Spielstands, die Darstellung der GUI sowie die Interaktion zwischen Benutzeroberfläche und Spiellogik überprüft.

5 Implementierungsphase

5.1 Implementierung der Benutzeroberfläche

Die grafische Benutzeroberfläche wurde in Java entwickelt und in das bestehende Projekt integriert. Ziel war es, die vorhandene Konsolenanwendung durch eine intuitive Desktop-Oberfläche zu ersetzen, ohne die bestehende Spiellogik grundlegend zu verändern.

Die Anwendung ist in mehrere Bereiche unterteilt. Das Hauptmenü bietet die Funktionen zum Starten eines neuen Spiels, Laden eines Spielstands, Öffnen des Optionsmenüs sowie zum Beenden der Anwendung.

Während des Spielverlaufs werden die wichtigsten Informationen, wie der aktuelle Spielerstatus, das Inventar und verfügbare Aktionen, übersichtlich dargestellt. Benutzereingaben erfolgen über Schaltflächen und Dialogfenster anstelle von Texteingaben.

Die Oberfläche wurde schrittweise implementiert und nach jeder Erweiterung auf ihre Funktionalität überprüft. Screenshots der fertigen Anwendung befinden sich im Anhang A.3: Screenshots der Anwendung auf Seite v.

5.2 Implementierung der Geschäftslogik

Die bestehende Spiellogik wurde weitgehend unverändert übernommen. Die Implementierung konzentrierte sich darauf, die vorhandenen Funktionen an die grafische Benutzeroberfläche anzubinden.

Die Verarbeitung von Benutzereingaben erfolgt über einen Controller, der Ereignisse aus der GUI entgegennimmt und die entsprechenden Methoden der Spiellogik aufruft. Nach Abschluss einer Aktion werden die aktuellen Spielinformationen an die Benutzeroberfläche übergeben und dort aktualisiert.

Zur Vermeidung einer engen Kopplung zwischen Oberfläche und Spiellogik wurde das MVC-Architekturprinzip umgesetzt. Dadurch bleiben Darstellung und Geschäftslogik voneinander getrennt und können unabhängig weiterentwickelt werden.

Im Rahmen der Implementierung wurden unter anderem folgende Funktionen realisiert:

- Starten eines neuen Spiels
- Laden und Speichern eines Spielstands
- Anzeige des Spielerstatus

- Darstellung des Inventars
- Durchführung von Kämpfen über die GUI
- Aktualisierung der Benutzeroberfläche nach Spielaktionen

Ausgewählte Quelltextausschnitte, welche die Kommunikation zwischen Benutzeroberfläche und Spiellogik sowie die Verarbeitung von Benutzeraktionen verdeutlichen, befinden sich im Anhang.

Die Klasse `InventoryScreen` ist beispielhaft in Anhang A.5: Klasse: `InventoryScreen` auf Seite xv dargestellt.

6 Dokumentation

6.1 Entwicklerdokumentation

Zur Dokumentation des Quellcodes wurde für sämtliche Klassen eine Javadoc-Dokumentation erstellt. Diese beschreibt die Aufgaben der einzelnen Klassen sowie die verwendeten Methoden, Parameter und Rückgabewerte. Dadurch wird die Wartbarkeit des Projekts verbessert und zukünftige Erweiterungen der Anwendung erleichtert.

Ergänzend wurde die Architektur der Anwendung mithilfe von UML-Diagrammen dokumentiert. Das Klassendiagramm stellt die Struktur der Anwendung dar, während ein Sequenzdiagramm den Ablauf eines typischen Spielzuges zwischen Benutzeroberfläche, Controller und Spiellogik veranschaulicht.

Die Entwicklerdokumentation wurde mittels Javadoc automatisch generiert. Ein beispielhafter Auszug aus der Dokumentation einer Klasse findet sich im Anhang A.4: Entwicklerdokumentation auf Seite ix.

7 Fazit

7.1 Soll-/Ist-Vergleich

Das Ziel des Projekts bestand darin, die bestehende Konsolenanwendung „Yggdrasil’s Depths“ um eine grafische Benutzeroberfläche zu erweitern. Dieses Ziel wurde erreicht. Die entwickelte GUI ermöglicht eine intuitive Bedienung der Anwendung und nutzt dabei die vorhandene Spiellogik, ohne deren grundlegende Funktionsweise zu verändern.

7 Fazit

Da es sich um ein eigenständig durchgeführtes Lernprojekt handelt, erfolgte die Abnahme durch den Auftraggeber, der zugleich der Entwickler des Projekts ist. Die definierten Anforderungen konnten vollständig umgesetzt werden.

Die ursprüngliche Projektplanung konnte weitgehend eingehalten werden. Während der Implementierung zeigte sich, dass die Anbindung der grafischen Benutzeroberfläche an die bestehende Spiellogik einen etwas höheren Aufwand erforderte als ursprünglich geplant. Dieser Mehraufwand konnte jedoch durch einen geringeren Aufwand in anderen Projektphasen ausgeglichen werden, sodass die geplante Gesamtprojektdauer von 80 Stunden eingehalten wurde.

Wie in Tabelle 3 zu erkennen ist, konnte die Zeitplanung bis auf wenige Ausnahmen eingehalten werden.

Phase	Geplant	Tatsächlich	Differenz
Analysephase	6 h	6 h	±0 h
Entwurfsphase	12 h	8 h	+4 h
Implementierungsphase	40 h	48 h	-8 h
Erstellen der Dokumentation	22 h	18 h	+4 h
Gesamt	80 h	80 h	

Tabelle 3: Soll-/Ist-Vergleich

7.2 Lessons Learned

Durch das Projekt konnten die im Unterricht vermittelten Kenntnisse zur Entwicklung grafischer Benutzeroberflächen praktisch angewendet und vertieft werden. Insbesondere die Umsetzung des MVC-Architekturprinzips verdeutlichte die Vorteile einer klaren Trennung zwischen Benutzeroberfläche, Steuerung und Spiellogik.

Darüber hinaus zeigte sich, dass die frühzeitige Planung der Benutzeroberfläche sowie regelmäßige Funktionstests während der Implementierung dazu beitrugen, Fehler frühzeitig zu erkennen und den Entwicklungsaufwand zu reduzieren. Außerdem wurden praktische Erfahrungen im Umgang mit Java, der GUI-Entwicklung sowie der strukturierten Projektdokumentation gesammelt.

7.3 Ausblick

Die entwickelte grafische Benutzeroberfläche bildet eine Grundlage für zukünftige Erweiterungen des Projekts. Denkbare Weiterentwicklungen sind beispielsweise die Ergänzung weiterer Kom-

7 Fazit

fortfunktionen innerhalb der Benutzeroberfläche, die Verbesserung des visuellen Designs oder die Integration zusätzlicher Spielinhalte.

Darüber hinaus könnte zukünftig eine Anbindung an eine Datenbank zur Verwaltung von Spielständen oder eine plattformunabhängige Bereitstellung der Anwendung umgesetzt werden. Aufgrund der gewählten MVC-Architektur lassen sich solche Erweiterungen mit vergleichsweise geringem Anpassungsaufwand realisieren.

A Anhang

A.1 Detaillierte Zeitplanung

Analysephase	6 h
1. Analyse des Ist-Zustands	3 h
2. Wirtschaftlichkeitsanalyse	1 h
3. Erstellen eines Anwendungsfalldiagramm	2 h
Entwurfsphase	12 h
1. Erstellen der GUI-Mock-ups	12 h
Implementierungsphase	40 h
1. Erstellen des Hauptmenüs	10 h
2. Erstellen des Optionsmenüs	6 h
3. Erstellen der Explorationsansicht	10 h
4. Erstellen der Kampfansicht	9 h
5. Testen und Fehlerbehebung	5 h
Erstellen der Dokumentation	22 h
1. Erstellen der Projektdokumentation	8 h
2. Erstellen der Javadoc	3 h
3. Erstellen der Projektpräsentation	8 h
4. Vorbereitung auf die Projektpräsentation	3 h
Gesamt	80 h

A.2 Oberflächenentwürfe

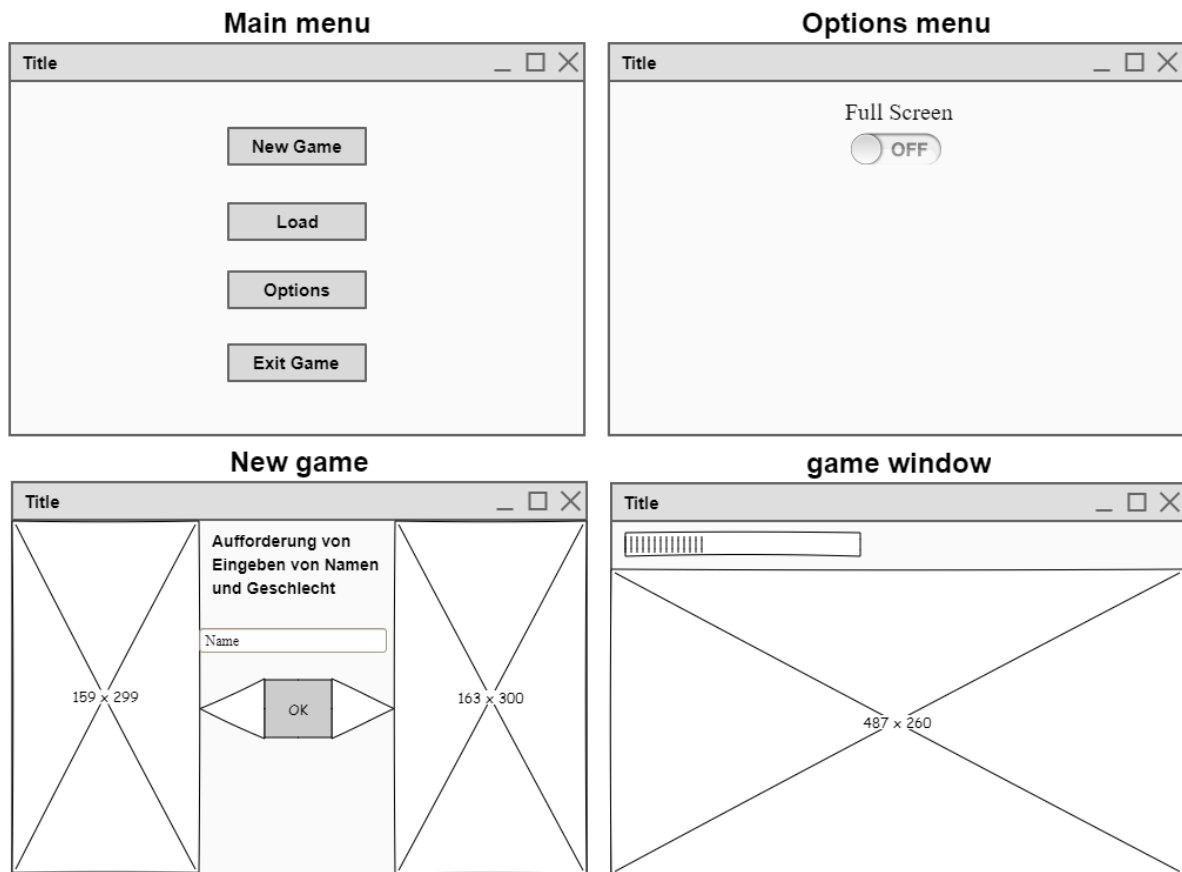


Abbildung 2: GUI-Mock-ups des Hauptmenüs, des Optionsmenüs, des Fensters zur Erstellung eines neuen Spiels sowie des Spielfensters.

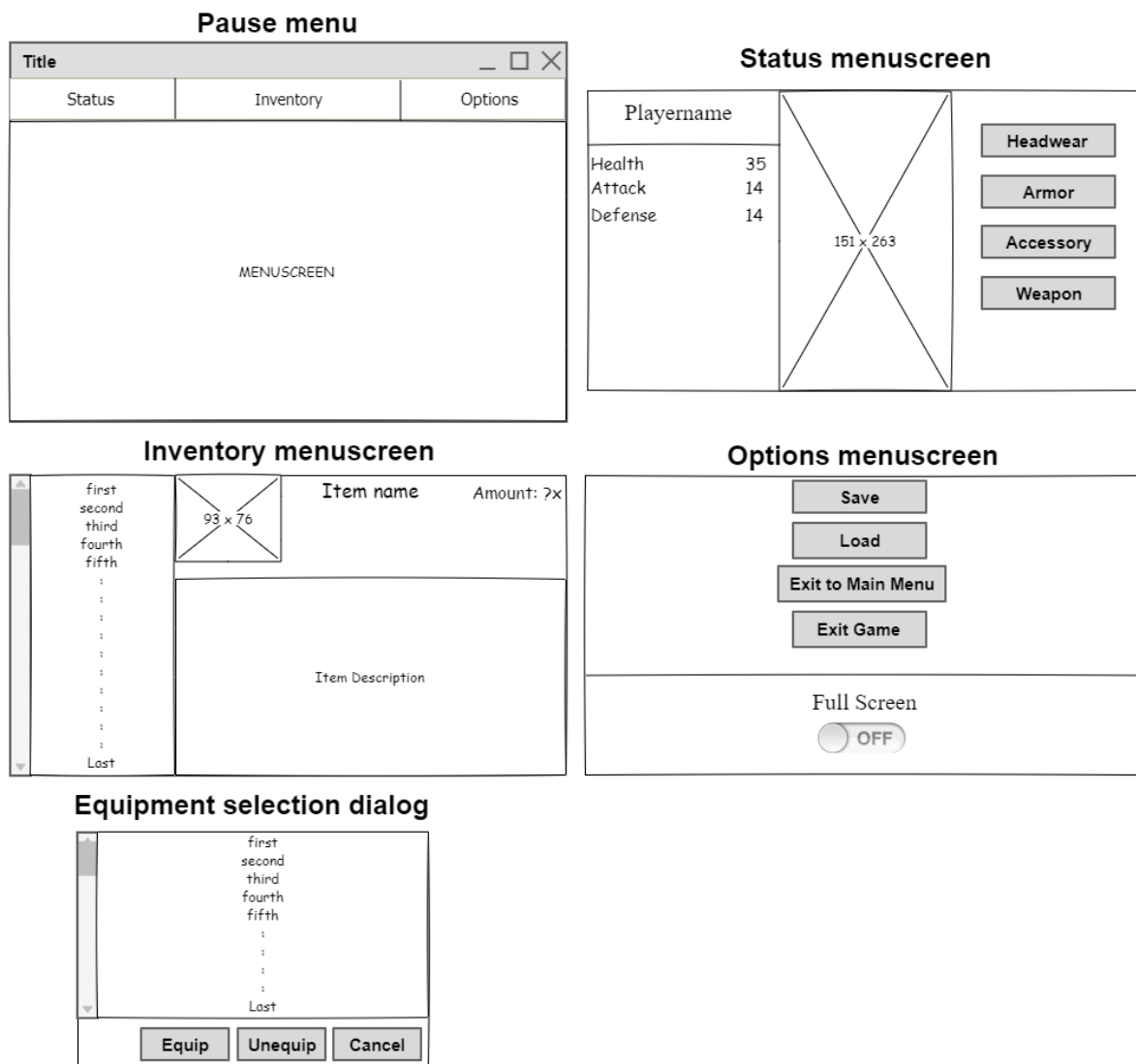


Abbildung 3: GUI-Mock-ups des Pausenmenüs und seiner Untermenüs: Status, Inventar, Optionen sowie Ausrüstungsauswahl.

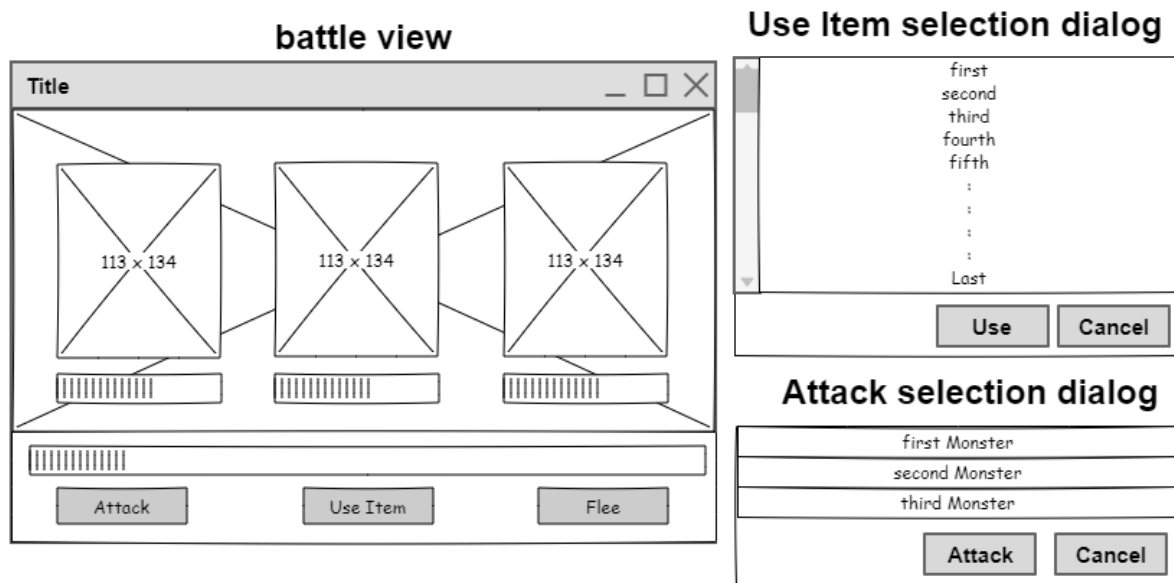


Abbildung 4: GUI-Mock-ups der Kampfansicht sowie des Angriffs- und des Itemauswahlmenüs.

A.3 Screenshots der Anwendung

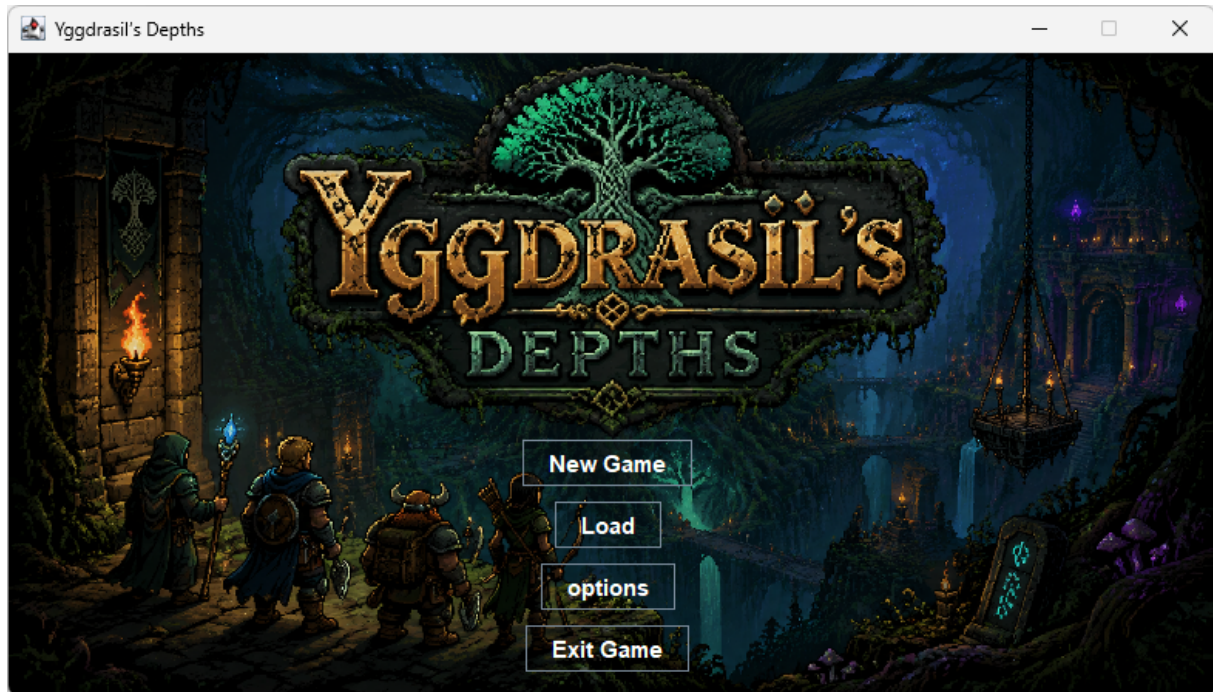


Abbildung 5: Hauptmenü

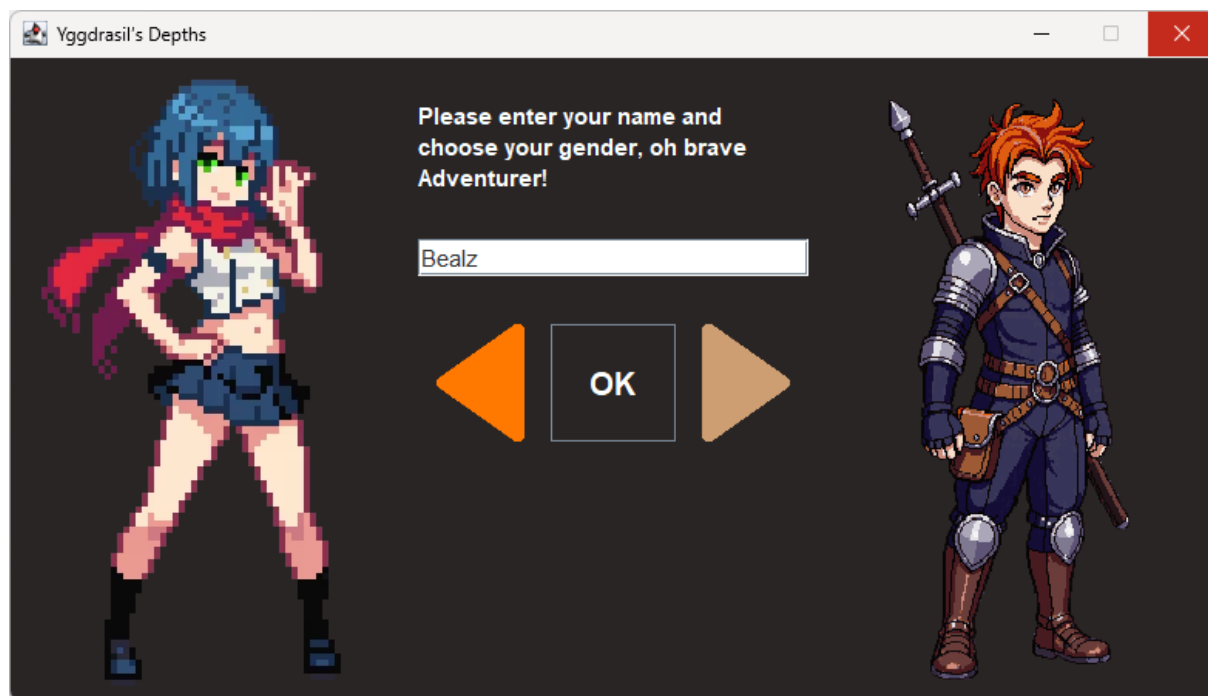


Abbildung 6: Fenster zur Erstellung eines neuen Spiels



Abbildung 7: Spielfenster während der Bewegung im Dungeon

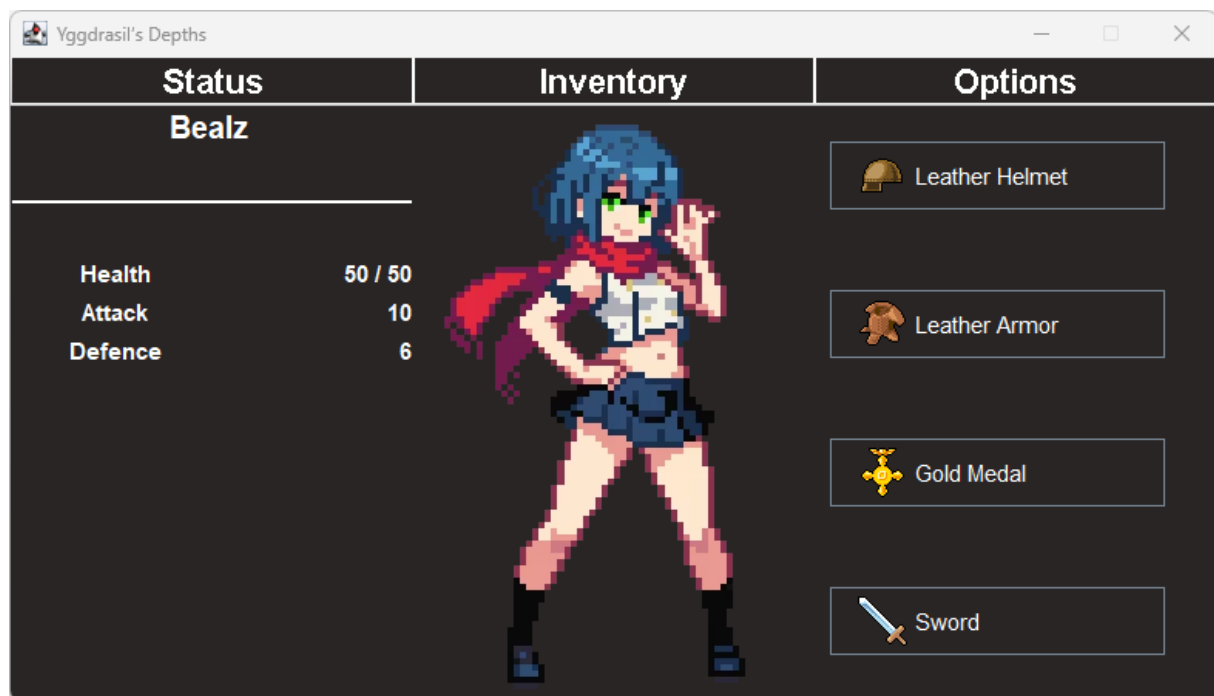


Abbildung 8: Status des Spielers



Abbildung 9: Inventar des Spielers

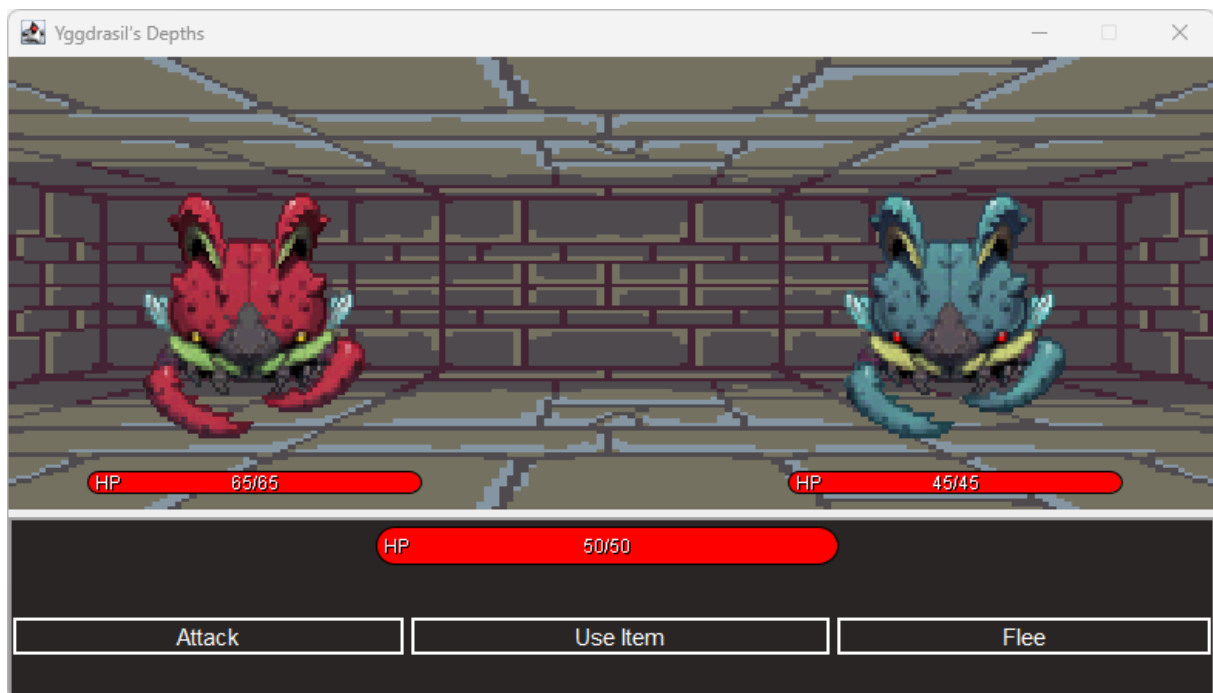


Abbildung 10: Spielfenster während eines Kampfes

A.4 Entwicklerdokumentation

Package `model.entity`

Klasse **Player**

`java.lang.Object`
`model.entity.Entity`
`model.entity.Player`

Alle implementierten Schnittstellen:

`Serializable`

```
public class Player
extends Entity
implements Serializable
```

Represents the player character in the game.

The player can move through a dungeon, manage an inventory, equip items, and engage in combat with enemies. Movement is based on a grid-based dungeon system with directional exits.

Seit:

2026

Autor:

Bastian Rentzsch

Siehe auch:

[Serialisierte Form](#)

Konstruktorübersicht

Konstruktor

Konstruktor	Beschreibung
<code>Player(String name, int health, int x, int y, String gender, Dungeon dungeon)</code>	Creates a new player in the given dungeon at the specified position.

Methodenübersicht

Alle Methoden

Instanzmethoden

Konkrete Methoden

Modifizierer und Typ	Methode	Beschreibung
void	<code>attack(Enemy enemy)</code>	Attacks the specified enemy, calculating damage based on weapon and enemy defense.

void	equip (Item item)	Equips an item if it is equippable.
int	getDamage ()	Calculates the player's total attack damage based on equipped weapons.
int	getDefense ()	Calculates the player's total defense based on equipped armor pieces.
Direction	getDirFlee ()	Returns the opposite direction of the last movement direction.
Dungeon	getDungeon ()	Returns the dungeon the player is currently in.
Item	getEquipment (EquipmentSlot equipmentSlot)	Returns the currently equipped item in the specified slot.
Direction	getFacing ()	Returns the direction the player is currently facing.
String ²	getGender ()	Returns the player's gender.
Inventory	getInventory ()	Returns the player's inventory.
int	getX ()	Returns the player's x-coordinate in the dungeon.
int	getY ()	Returns the player's y-coordinate in the dungeon.
void	move (Direction direction)	Moves the player in the specified direction if possible.
void	setCoordinates (int x, int y)	Updates the player's coordinates by the given delta values.
void	turnLeft ()	Rotates the player 90 degrees to the left.
void	turnRight ()	Rotates the player 90 degrees to the right.
void	unequip (EquipmentSlot slot)	Unequips an item from the specified slot and returns it to the inventory.

Von Klasse geerbte Methoden `model.entity.Entity`

`getHealth`, `getMaxHealth`, `getName`, `heal`, `setHealth`, `setMaxHealth`,
`takeDamage`

Von Klasse geerbte Methoden `java.lang.Object`

`equals`, `getClass`, `hashCode`, `notify`, `notifyAll`, `toString`, `wait`,
`wait`, `wait`

Konstruktordetails

Player

```
public Player(String name,  
              int health,  
              int x,  
              int y,  
              String gender,  
              Dungeon dungeon)
```

Creates a new player in the given dungeon at the specified position.

Parameter:

`name` - the player's name

`health` - the initial and maximum health

`x` - the starting x-coordinate in the dungeon

`y` - the starting y-coordinate in the dungeon

`gender` - the gender of the player

`dungeon` - the dungeon the player is placed in

Methodendetails

`getX`

```
public int getX()
```

Returns the player's x-coordinate in the dungeon.

Gibt zurück:

the x position

`getY`

```
public int getY()
```

Returns the player's y-coordinate in the dungeon.

Gibt zurück:

the y position

getGender

```
public String getGender()
```

Returns the player's gender.

Gibt zurück:

the gender string

getDungeon

```
public Dungeon getDungeon()
```

Returns the dungeon the player is currently in.

Gibt zurück:

the dungeon instance

getFacing

```
public Direction getFacing()
```

Returns the direction the player is currently facing.

Gibt zurück:

the facing direction

getDirFlee

```
public Direction getDirFlee()
```

Returns the opposite direction of the last movement direction. Useful for fleeing mechanics.

Gibt zurück:

the opposite direction of the last move

getDamage

```
public int getDamage()
```

Calculates the player's total attack damage based on equipped weapons.

Gibt zurück:

the total damage value

getDefense

```
public int getDefense()
```

Calculates the player's total defense based on equipped armor pieces.

Gibt zurück:

the total defense value

getEquipment

```
public Item getEquipment(EquipmentSlot equipmentSlot)
```

Returns the currently equipped item in the specified slot.

Parameter:

equipmentSlot - the slot to check

Gibt zurück:

the equipped item, or null if empty

getInventory

```
public Inventory getInventory()
```

Returns the player's inventory.

Gibt zurück:

the inventory instance

setCoordinates

```
public void setCoordinates(int x,  
                           int y)
```

Updates the player's coordinates by the given delta values.

Parameter:

x - change in x direction

y - change in y direction

equip

```
public void equip(Item item)
```

Equips an item if it is equippable.

If an item is already equipped in the same slot, it is moved back to the inventory.

Parameter:

item - the item to equip

unequip

```
public void unequip(EquipmentSlot slot)
```

Unequips an item from the specified slot and returns it to the inventory.

Parameter:

slot - the equipment slot to clear

move

```
public void move(Direction direction)
```

Moves the player in the specified direction if possible.

Movement is only performed if an exit exists in that direction. The player's position and current room are updated accordingly.

Parameter:

direction - the direction to move in

turnLeft

```
public void turnLeft()
```

Rotates the player 90 degrees to the left.

turnRight

```
public void turnRight()
```

Rotates the player 90 degrees to the right.

attack

```
public void attack(Enemy enemy)
```

Attacks the specified enemy, calculating damage based on weapon and enemy defense.

Parameter:

enemy - the enemy to attack

A.5 Klasse: InventoryScreen

Kommentare und simple Getter/Setter werden nicht angezeigt.

```
public class InventoryScreen extends JPanel {
    public InventoryScreen(Game game) {
        this.setLayout(new BorderLayout());

        JSplitPane splitPane = new JSplitPane();
        splitPane.setBackground(new Color(41, 37, 36));
        splitPane.setResizeWeight(0.1);
        splitPane.setDividerSize(0);
        splitPane.setBorder(null);
        splitPane.setEnabled(false);
        this.add(splitPane, BorderLayout.CENTER);

        JScrollPane scrollPane = new JScrollPane();
        scrollPane.setVerticalScrollBarPolicy (ScrollPaneConstants.VERTICAL_SCROLLBAR_ALWAYS);
        scrollPane.setVerticalScrollBar().setUI(new ScrollBarUI());
        scrollPane.setHorizontalScrollBarPolicy (ScrollPaneConstants.
            HORIZONTAL_SCROLLBAR_NEVER);
        scrollPane.setComponentOrientation(ComponentOrientation.RIGHT_TO_LEFT);
        splitPane.setLeftComponent(scrollPane);

        JPanel detailsPanel = new JPanel();
        detailsPanel.setForeground(new Color(255, 255, 255));
        detailsPanel.setBackground(new Color(41, 37, 36));
        splitPane.setRightComponent(detailsPanel);
        GridBagLayout gbl_detailsPanel = new GridBagLayout();
        gbl_detailsPanel.columnWidths = new int[]{156, 133, 80, 0};
        gbl_detailsPanel.rowHeights = new int[]{51, 224, 0};
        gbl_detailsPanel.columnWeights = new double[]{1.0, 1.0, 1.0, Double.MIN_VALUE};
        gbl_detailsPanel.rowWeights = new double[]{0.0, 1.0, Double.MIN_VALUE};
        detailsPanel.setLayout(gbl_detailsPanel);

        JLabel lblItemImage = new JLabel("");
        GridBagConstraints gbc_lblItemImage = new GridBagConstraints();
        gbc_lblItemImage.fill = GridBagConstraints.BOTH;
        gbc_lblItemImage.insets = new Insets(0, 0, 5, 5);
        gbc_lblItemImage.gridx = 0;
        gbc_lblItemImage.gridy = 0;
        detailsPanel.add(lblItemImage, gbc_lblItemImage);

        JLabel lblItemname = new JLabel("");
        lblItemname.setForeground(new Color(255, 255, 255));
        lblItemname.setBackground(new Color(255, 255, 255));
        lblItemname.setFont(new Font("SansSerif", Font.BOLD, 15));
```

```

GridBagConstraints gbc_lblItemname = new GridBagConstraints();
gbc_lblItemname.anchor = GridBagConstraints.NORTH;
gbc_lblItemname.fill = GridBagConstraints.HORIZONTAL;
gbc_lblItemname.insets = new Insets(0, 0, 5, 5);
gbc_lblItemname.gridx = 1;
gbc_lblItemname.gridy = 0;
detailsPanel.add(lblItemname, gbc_lblItemname);

JLabel lblItemAmount = new JLabel("");
lblItemAmount.setFont(new Font("SansSerif", Font.PLAIN, 13));
lblItemAmount.setForeground(new Color(255, 255, 255));
GridBagConstraints gbc_lblItemAmount = new GridBagConstraints();
gbc_lblItemAmount.anchor = GridBagConstraints.NORTHEAST;
gbc_lblItemAmount.insets = new Insets(0, 0, 5, 0);
gbc_lblItemAmount.gridx = 2;
gbc_lblItemAmount.gridy = 0;
detailsPanel.add(lblItemAmount, gbc_lblItemAmount);

JTextPane textDescription = new JTextPane();
textDescription.setContentType("text/html");
textDescription.setEnabled(false);
textDescription.setFont(new Font("SansSerif", Font.PLAIN, 15));
textDescription.setBackground(new Color(41, 37, 36));
textDescription.setForeground(new Color(255, 255, 255));
textDescription.setEditable(false);
GridBagConstraints gbc_textDescription = new GridBagConstraints();
gbc_textDescription.fill = GridBagConstraints.BOTH;
gbc_textDescription.gridwidth = 3;
gbc_textDescription.gridx = 0;
gbc_textDescription.gridy = 1;
detailsPanel.add(textDescription, gbc_textDescription);

JList<String> list = new JList<>();
list.setForeground(new Color(255, 255, 255));
list.setBackground(new Color(41, 37, 36));
list.setFont(new Font("SansSerif", Font.PLAIN, 15));
list.setSelectionMode(ListSelectionModel.SINGLE_SELECTION);
scrollPane.setViewportView(list);

List<String> entries = new ArrayList<>();
for (Map.Entry<Item, Integer> e : game.player.getInventory().getItems().entrySet()) {
    entries.add(e.getKey().getName());
}

list.setModel(new AbstractListModel<>() {
    public int getSize() {
        return entries.size();
    }
});

```

```

    }

    public String getElementAt(int index) {
        return entries.get(index);
    }
});
list.addListSelectionListener(new ListSelectionListener() {
    public void valueChanged(ListSelectionEvent e) {
        if (!e.getValueIsAdjusting()) {
            String selected = list.getSelectedValue();
            if (selected != null) {
                Map.Entry<Item, Integer> result = game.player.getInventory().getItem(selected);

                if (result != null) {
                    Item item = result.getKey();
                    int amount = result.getValue();

                    String imagepath = "./res/images/items/";

                    if (item.getType() == Itemtype.HEADWEAR) {
                        imagepath += "headwear/" + item.getImageName();
                    }
                    else if (item.getType() == Itemtype.ARMOR) {
                        imagepath += "armor/" + item.getImageName();
                    }
                    else if (item.getType() == Itemtype.ACCESSORY) {
                        imagepath += "accessory/" + item.getImageName();
                    }
                    else if (item.getType() == Itemtype.WEAPON) {
                        imagepath += "weapon/" + item.getImageName();
                    }
                    else if (item.getType() == Itemtype.CONSUMABLES) {
                        imagepath += "consumables/" + item.getImageName();
                    }
                    else {
                        imagepath += item.getImageName();
                    }

                    ImageIcon icon = new ImageIcon(imagepath);
                    lblItemImage.setIcon(icon);

                    lblItemname.setText(item.getName());
                    lblItemAmount.setText("Amount: " + amount + "x");
                    textDescription.setText(item.getDescription());
                }
            }
        }
    }
});

```

```
    }  
    });  
  
    UIManager.put("List.focusCellHighlightBorder", BorderFactory.createEmptyBorder());  
  
    list.setSelectionBackground(new Color(255, 120, 0));  
    list.setSelectionForeground(Color.WHITE);  
  
    if (entries.size() > 0) {  
        list.setSelectedIndex(0);  
    }  
    }  
}
```

Listing 1: Klasse: InventoryScreen

A.6 Klassendiagramm der grafischen Benutzeroberfläche

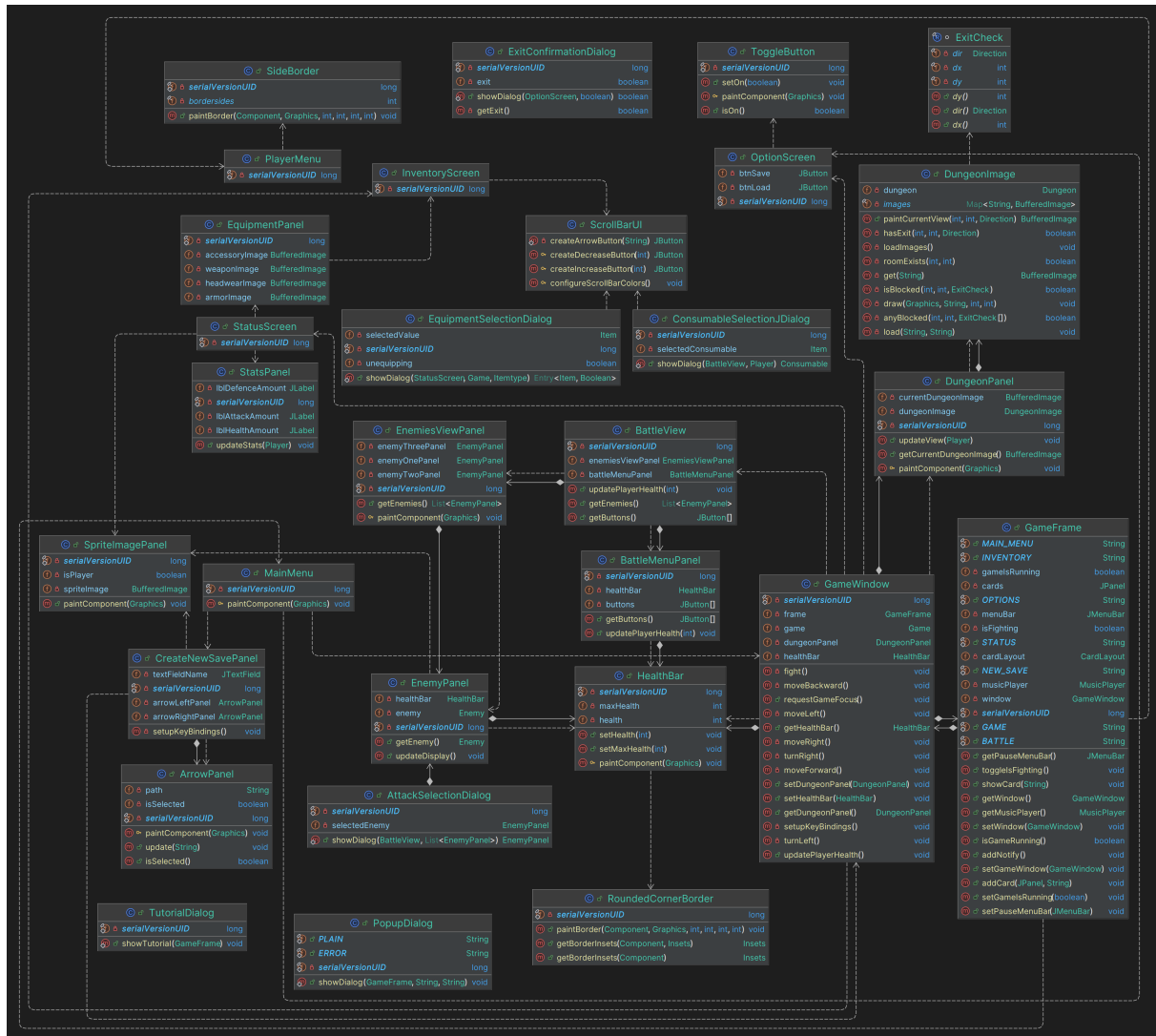


Abbildung 11: Klassendiagramm der grafischen Oberfläche

A.7 Sequenzdiagramm

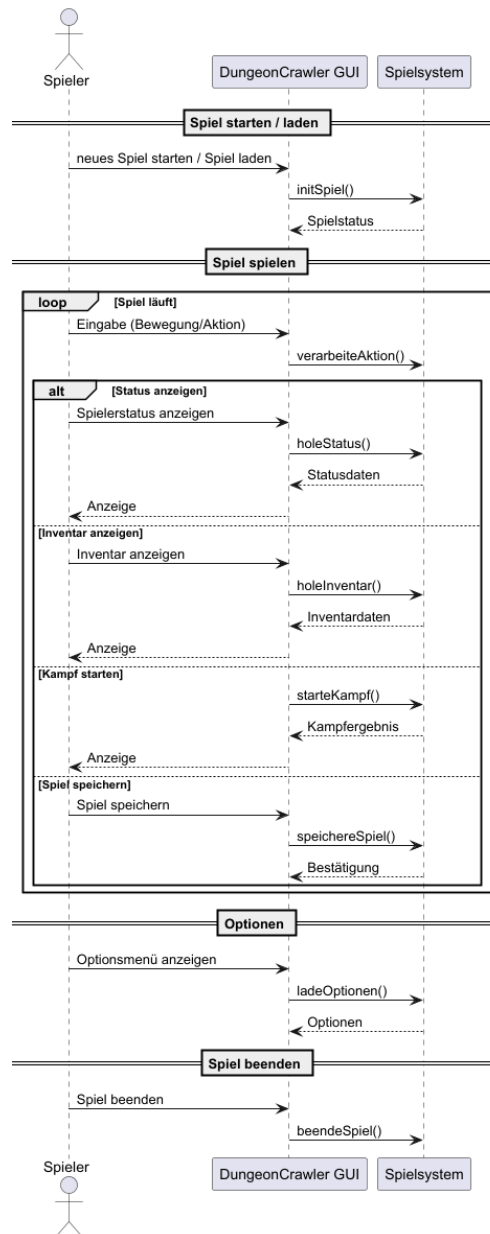


Abbildung 12: Sequenzdiagramm